SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 –DAST SIMULATION

Course Code:	CSA5803	**Course Title:**	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 2
Weightage:	30% of CIA	**Total Marks:**	100
CO3 Statement:	Simulate Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA).		
Student Name:	**Syed Saniya Iqbal**	**Reg. No.:**	**192465043**
Section / Batch:	**CSE-Cybersecurity**	**Date:**	**14/8/26**

Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

Question Type	Focus Area	Questions	Marks
DAST SIMULATION	Application based	Q1	Q1 –100
	GRAND TOTAL:	**100 Marks**	

Code of Conduct

*I **Syed Saniya Iqbal(192465043)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student:***Syed Saniya Iqbal(192465043)***

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA)

If your previous project was **SAST simulation for an online banking application**, you can extend it into a broader application-security simulation with **DAST + SCA**.

---

1. Dynamic Application Security Testing (DAST)

What is DAST?

DAST is a security testing method that analyzes a **running web application** from the outside.

Unlike SAST, which examines source code, DAST interacts with the application like an attacker and looks for vulnerabilities in its behavior.

Simple Difference

SAST	DAST
Tests source code	Tests running application
Static analysis	Dynamic analysis
No execution required	Application must be running
Finds coding weaknesses	Finds runtime/web vulnerabilities
White-box approach	Mostly black-box approach

---

2. DAST Simulation Scenario

Scenario: Online Banking Web Application

A synthetic online banking application is deployed locally.

Example:

http://localhost:5000

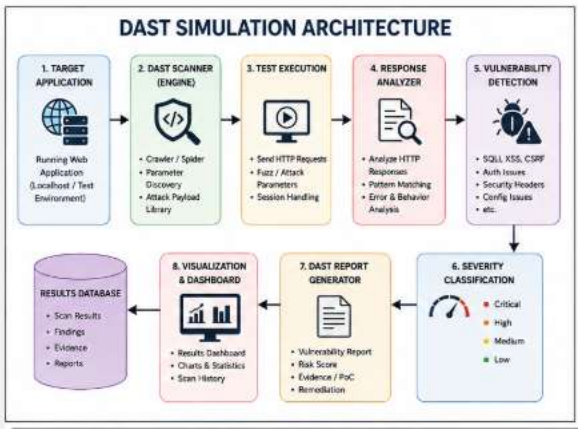
The DAST simulator sends different requests to the application and analyzes the responses.

Application Modules

Module	Example Function
Login	User authentication
Dashboard	Account information
Balance	View balance
Transfer	Transfer money
Profile	Update user information
Transactions	View transaction history
Logout	End session

---

3. DAST Architecture



---

4. DAST Security Tests

The simulator can perform controlled tests for:

Test	What It Checks	Severity
SQL Injection	Unsafe database input handling	Critical
XSS	Unsafe user input rendering	High
Authentication	Weak login controls	High
Session Security	Improper session handling	High
Security Headers	Missing HTTP security headers	Medium
Input Validation	Invalid input handling	Medium
HTTPS	Insecure communication	High
Error Handling	Sensitive information in errors	Medium
Access Control	Unauthorized resource access	Critical
Cookie Security	Missing Secure/HttpOnly flags	Medium

---

5. DAST Simulation Workflow

START
↓
Start Web Application
↓
Discover Application Pages
↓
Identify Input Parameters
↓
Send Test Requests
↓
Capture HTTP Responses
↓
Analyze Responses
↓
Detect Vulnerability?
- NO → Continue Testing
- YES
  ↓
  Create Finding

↓
Assign Severity
↓
Generate Report
↓
END

---

6. Example DAST Test

Suppose the application has:

/login?username=test

The simulator can test controlled input such as:

test' OR '1'='1

The simulator then analyzes the application's response.

For example:

Test ID : DAST-001
Target : /login
Parameter : username
Test Type : SQL Injection
Result : Suspicious Response
Severity : Critical
Status : Vulnerable

Important: In an educational simulator, you can use synthetic responses rather than attacking a real application.

---

7. DAST Sample Report

ID	URL	Vulnerability	Severity	Status
D001	/login	SQL Injection	Critical	Open
D002	/profile	Reflected XSS	High	Open
D003	/login	Weak Authentication	High	Open
D004	/transfer	Missing Input Validation	Medium	Open
D005	/	Missing Security Header	Medium	Open

---

8. DAST Dashboard

Your simulator can display:

DAST SECURITY SCAN

URLs Scanned : 12
Requests Sent : 85
Vulnerabilities : 8
Critical : 2
High : 3

```
Medium          : 3
Low             : 0
Security Score  : 62/100
Risk Level      : MODERATE
```

---

9. Software Composition Analysis (SCA)

What is SCA?

Software Composition Analysis (SCA) identifies security vulnerabilities and risks in **third-party libraries, packages, frameworks, and open-source dependencies** used by an application.

For example, a Python application may use:

Flask
requests
SQLAlchemy
cryptography
PyMySQL

The SCA simulator checks these dependencies against a **synthetic vulnerability database**.

---

10. Why SCA is Required

Modern applications rarely consist entirely of code written by the development team.

For example:

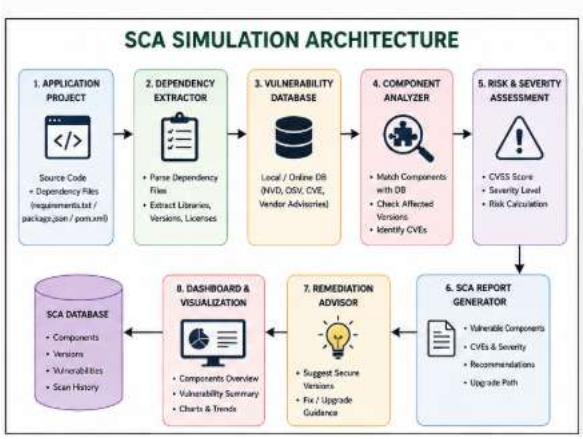
Application

```
├── Flask
├── Requests
├── SQLAlchemy
├── PyMySQL
└── Cryptography
```

If one dependency contains a known vulnerability, the application may become vulnerable even if the developer's own code is secure.

---

11. SCA Architecture



---

12. Synthetic SCA Dataset

For your simulation, you can create an artificial dependency dataset.

ID	Package	Version	Vulnerability	Severity
SCA001	Flask	1.0.2	Example vulnerability	High
SCA002	Requests	2.19.0	Example vulnerability	Medium
SCA003	SQLAlchemy	1.3.0	Example vulnerability	High
SCA004	PyMySQL	0.9.0	No known issue	Safe
SCA005	Cryptography	2.5	Example vulnerability	Critical

These are **simulation values**, not claims that these specific versions currently have those vulnerabilities.

---

13. SCA Simulation Workflow

START
↓
Load Application
↓
Read Dependency File
↓
Extract Package Names
↓
Extract Package Versions
↓
Compare With Vulnerability Database
↓
Vulnerability Found?
├── NO → Mark Dependency SAFE
└── YES
↓
Create Finding
↓
Assign Severity
↓

Recommend Upgrade
↓
Generate SCA Report
↓
END

---

14. Example SCA Finding

Finding ID : SCA-001
Package : ExamplePackage
Current Version: 1.2.0
Vulnerability : Known Security Issue
Severity : High
Status : Vulnerable

Recommended Action:
Upgrade to a secure supported version.

---

15. SCA Report

Package	Current Version	Severity	Status	Recommendation
Flask	1.0.2	High	Vulnerable	Upgrade
Requests	2.19.0	Medium	Vulnerable	Upgrade
SQLAlchemy	1.3.0	High	Vulnerable	Upgrade
PyMySQL	0.9.0	Safe	Safe	No action
Cryptography	2.5	Critical	Vulnerable	Upgrade immediately

---

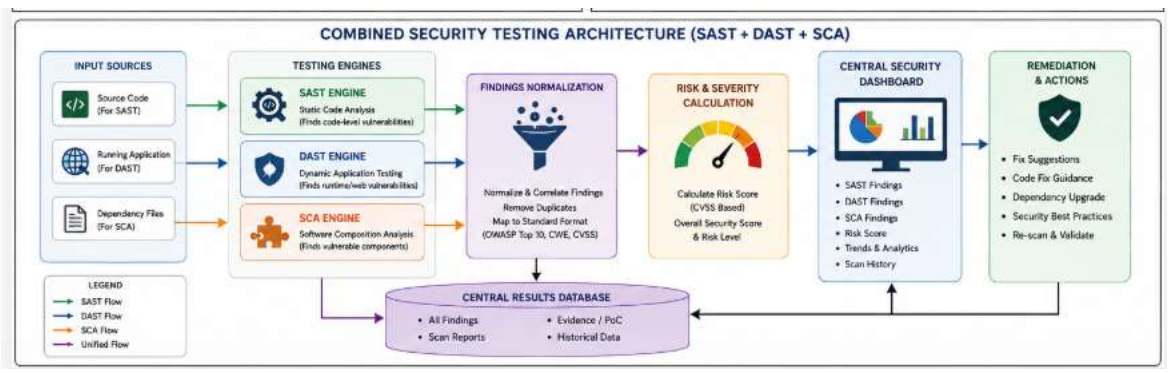
16. DAST vs SCA

Feature	DAST	SCA
Full Name	Dynamic Application Security Testing	Software Composition Analysis
Tests	Running application	Dependencies
Input	HTTP requests	Package/dependency files
Main Target	Web application behavior	Third-party components
Requires Running App	Yes	No
Finds	XSS, SQLi, auth issues, headers, etc.	Vulnerable libraries/packages
Analysis Type	Dynamic	Dependency analysis
Output	DAST report	Dependency vulnerability report

---

17. SAST + DAST + SCA Combined Architecture

Since you already have the **SAST simulation**, the strongest project would combine all three:



Final Project Structure

You can present the complete project as:

Module 1 – SAST Simulation
Source-code vulnerability detection.

Module 2 – DAST Simulation
Runtime/web application vulnerability detection.

Module 3 – SCA Simulation
Third-party dependency vulnerability detection.

Module 4 – Unified Security Dashboard
Combines SAST + DAST + SCA results and generates an overall application security score.

This is a much stronger project than having SAST alone because it demonstrates **three different layers of application security testing**.